Zellic

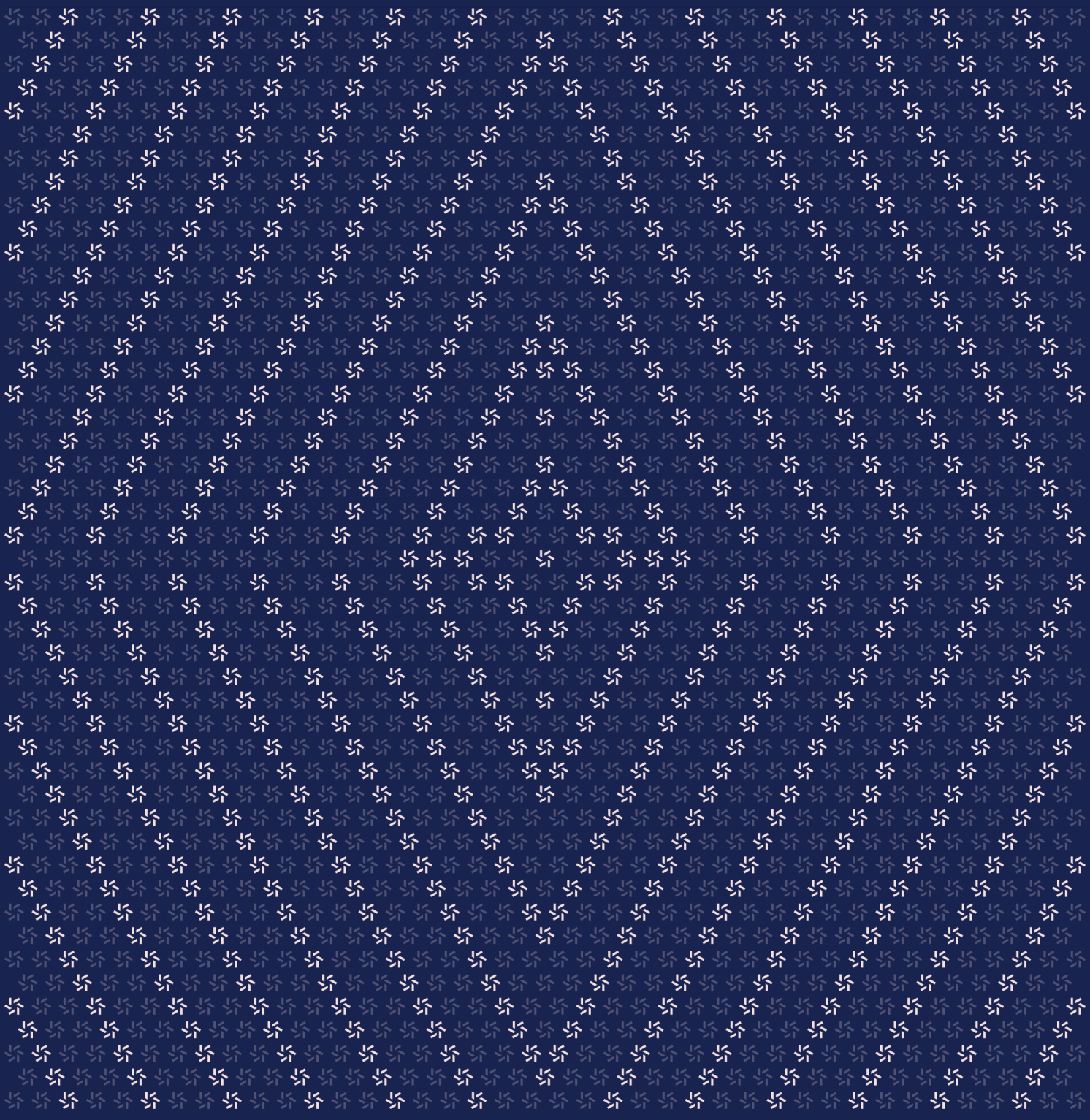
Prepared for
Brent Perreault
Arjun Sharma
Paxos

Prepared by
Katerina Belotskaia
Kuilin Li
Zellic

October 26, 2026

Bitmap Nonce Delegate

Smart Contract Security Assessment



Contents

About Zellic **3**

1. Overview **4**

1.1. Executive Summary 4

1.2. Goals of the Assessment 4

1.3. Non-goals and Limitations 4

1.4. Results 5

2. Introduction **6**

2.1. About Bitmap Nonce Delegate 6

2.2. Methodology 6

2.3. Scope 8

2.4. Project Overview 8

2.5. Project Timeline 9

3. Detailed Findings **10**

3.1. Delegate does not support ERC-721 and ERC-1155 receiver hooks 10

3.2. Zero gas limits still allow relayer-induced failures 12

3.3. Target revert data cannot be distinguished from the delegate's own errors 14

4. Assessment Results **16**

4.1. Disclaimer 16

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a follow-up security assessment for Paxos on October 23, 2026. During this engagement, Zellic reviewed Bitmap Nonce Delegate's code for security vulnerabilities, design issues, and general weaknesses in security posture.

This assessment follows our September 28, 2026 review of Bitmap Nonce Delegate. It covers the remediation of the previous findings and the other changes made to the contract since then.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can an authorization be replayed after its ID has been consumed, or used outside its intended account, chain, or execution context?
- Does the signature bind every action, recipient, asset, amount, gas limit, and operation ID that execution relies on?
- Can a malformed ID, a bitmap word boundary, a duplicate ID within a batch, or concurrent submission cause an operation to execute twice or consume the wrong ID?
- Are all IDs in a batch reserved and written to storage before the first external call, so a callback cannot replay or cancel a later ID in the same batch?
- Does a strict batch failure leave every ID available, and does a completed tolerant batch consume exactly the IDs of the operations it attempted?
- Can a relayer or call target make a capped operation run with less than its signed gas without reverting the whole batch?
- Does the packed target/gas-limit encoding always match what is signed, with the reserved bits rejected?
- Can a relayer execute anything beyond the signed batch?
- Do the changes fully address ZELLIC-1H7VJA, -3W9ECY, -SK7GFB, and -MKM32S, the findings from our previous report on Bitmap Nonce Delegate?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Off-chain signing, encoding, and relaying software
- Front-end components
- Infrastructure relating to the project

- Key custody

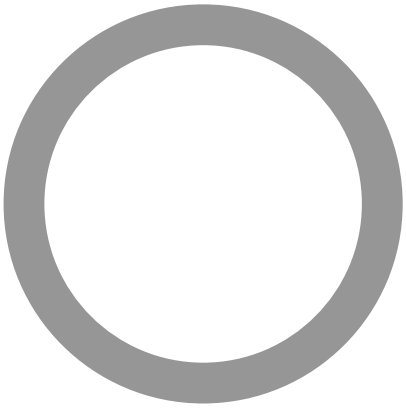
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Bitmap Nonce Delegate contracts, we discovered three findings, all of which were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	3



2. Introduction

2.1. About Bitmap Nonce Delegate

Paxos contributed the following description of Bitmap Nonce Delegate:

> Bitmap Nonce Delegate allows a holding account to authorize operations with EIP-712 signatures without relying on the network nonce for replay protection. Relayers submit those operations and pay network fees. One signature authorizes an ordered batch of operations. Each operation's ID selects one bit in a storage bitmap, and every ID in a batch is reserved before any call executes. A strict batch reverts entirely if any operation fails, which leaves every ID available. A tolerant batch continues past failures and consumes the IDs of every operation it attempted. Operations can carry a signed gas limit, so a call target can't change how much gas the following operations get. The holding account can also call itself directly to run an atomic multicall, which uses no signature or ID.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We

also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Bitmap Nonce Delegate Contracts

Type	Solidity
Platform	EVM-compatible
Target	paxos-token-contract-internal
Repository	https://github.com/paxosglobal/paxos-token-contract-internal ↗
Version	`b13929cb11d86d7c2f1e885574a71687c92882a9`
Programs	`contracts/delegates/BitmapNonceDelegate.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 0.3 person-weeks. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
Security Engineer
kate@zellic.io ↗

Kuilin Li
Security Engineer
kuilin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

October 23, 2026 Primary review period

3. Detailed Findings

3.1. Delegate does not support ERC-721 and ERC-1155 receiver hooks

Target	BitmapNonceDelegate	**ID**	ZELLIC-EHPJQX
Category	Integration Risks	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

After EIP-7702 delegation, the account's code is the 23-byte designator `0xef0100 || implementation`. ERC-721 and ERC-1155 token contracts check whether the recipient of a safe transfer has code and, if it does, call a receiver hook on it. For example, OpenZeppelin's ERC-721 implementation performs the following check in `safeTransferFrom` and `_safeMint`:

```
if (to.code.length > 0) {
    try IERC721Receiver(to).onERC721Received(operator, from, tokenId, data)
    returns (bytes4 retval) {
        if (retval != IERC721Receiver.onERC721Received.selector) {
            revert IERC721Errors.ERC721InvalidReceiver(to);
        }
    } catch (bytes memory reason) {
        // [...]
    }
}
```

ERC-1155 transfers call `onERC1155Received` or `onERC1155BatchReceived` in the same way. The delegate implements none of these hooks and has no `fallback` function, so the call reverts. As a result, safe transfers and safe mints of ERC-721 and ERC-1155 tokens to a delegated account revert, whereas they would have succeeded before the delegation. We confirmed this for OpenZeppelin's ERC-721 implementation in a Foundry test.

The documentation does not mention this limitation.

Impact

Third parties cannot deliver ERC-721 or ERC-1155 tokens to a delegated account through safe transfers or safe mints. The transfers revert, so no tokens are lost, but integrations that rely on them fail after the account is delegated.

Recommendations

If delegated accounts are expected to receive such tokens, the delegate should implement `onERC721Received`, `onERC1155Received`, and `onERC1155BatchReceived`. Otherwise, the limitation should be documented.

Remediation

This issue has been acknowledged by Paxos, and a fix was implemented in commit `0792f0ee`.

3.2. Zero gas limits still allow relayer-induced failures

Target	BitmapNonceDelegate	**ID**	ZELLIC-S39168
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The remediation of ZELLIC-3W9ECY added signed per-intent gas limits, but a gas limit of zero is still accepted. In `_callTarget`, such an intent skips the gas sufficiency check and forwards all available gas, as every call did in the originally reviewed version:

```
let gasLimit := and(shr(160, targetAndGasLimit), 0xffffffffffffffff)
switch gasLimit
case 0 { gasLimit := gas() }
default {
    if lt(gas(), add(add(gasLimit, div(gasLimit, 63)), gasReserve)) {
        mstore(0, insufficientGasSelector)
        revert(0, 4)
    }
}
ok := call(gasLimit, to, value, add(data, 0x20), mload(data), 0, 0)
```

When `revertOnFailure` is false, a failed call only emits `CallFailed`, and the intent's ID, which was reserved before the first call, stays consumed:

```
(bool ok, bytes memory ret) = _callTarget(intent.targetAndGasLimit,
    intent.value, intent.data, gasReserve);
if (!ok) {
    if (metadata.revertOnFailure) {
        assembly { revert(add(ret, 0x20), mload(ret)) }
    }
    emit CallFailed(digest, intent.nonce, ret);
}
```

A relayer can therefore send just enough gas for the uncapped call to run out, while the 1/64 that the EVM retains for the delegate lets the batch complete.

The documentation already describes this behavior and advises nonzero limits when the relayer is untrusted. However, the expected relayer is not necessarily the actual relayer on-chain. Since `MODE_SIGNED_BATCH` calls to `execute` are permissionless, the actual relayer may be a front-runner

instead of the party that signed the original transaction.

Impact

In a batch with `revertOnFailure` set to false, any party that submits the batch can make an intent with a zero gas limit fail by supplying just enough gas for the delegate to finish. The failed intent's ID is consumed, so the authority must sign a replacement. Because submission is permissionless, this applies even when the authority trusts the relayer it hands the batch to. No funds are at risk, and the authority can avoid the issue by always signing a nonzero gas limit.

Recommendations

We recommend that the contract reject a zero gas limit when `revertOnFailure` is false. In strict batches, a zero limit is harmless because a failed call reverts the whole batch.

Remediation

This issue has been acknowledged by Paxos, and a fix was implemented in commit `0792f0ee` ↗.

3.3. Target revert data cannot be distinguished from the delegate's own errors

Target	BitmapNonceDelegate	**ID**	ZELLIC-VPCPPR
Category	Integration Risks	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

When a call fails, the delegate passes the target's return data on unchanged. In a strict batch, it reverts with that data, and in a batch with `revertOnFailure` set to false, it records the data in `CallFailed`:

```
(bool ok, bytes memory ret) = _callTarget(intent.targetAndGasLimit,
    intent.value, intent.data, gasReserve);
if (!ok) {
    if (metadata.revertOnFailure) {
        assembly { revert(add(ret, 0x20), mload(ret)) }
    }
    emit CallFailed(digest, intent.nonce, ret);
}
```

The only change that `_callTarget` makes to the return data is for replies longer than 256 bytes, which it replaces with the four-byte selector of `ReturnDataTooLarge()`:

```
if gt(size, _MAX_RETURNDATA) {
    n := 0
    size := 4
    mstore(add(ret, 0x20), tooLarge)
}
```

The effect depends on the failure policy. In a strict batch, the delegate reverts with the target's data as its own revert data. A target can therefore produce any of the delegate's own errors, such as `NonceUsed()`, `InsufficientGas()`, `InvalidSigner()`, or `ReturnDataTooLarge()`, and the revert data cannot be distinguished on-chain from a genuine error raised by the delegate.

In a batch with `revertOnFailure` set to false, the data appears in `CallFailed`, which the delegate emits only for failed calls, so it cannot be confused with the delegate's own errors. The only ambiguous value is the `ReturnDataTooLarge()` sentinel, which the delegate itself writes into `returnData` for replies longer than 256 bytes.

For example, the authority may sign an ordinary transfer to a token whose `transfer` function

reverts with one of the delegate's errors:

```
contract SpoofingToken {
    function transfer(address, uint256) external pure returns (bool) {
        revert BitmapNonceDelegate.NonceUsed();
    }
}
```

Impact

A relayer or other integration whose retry logic depends on these selectors can be misled by a target. For example, it may drop a batch on `NonceUsed()` or keep adding gas on `InsufficientGas()`. The contract's own state is unaffected, and the bitmap always reflects which IDs were actually consumed.

Recommendations

We recommend documenting that the revert data of a strict batch comes from the target and can match any of the delegate's own errors, and that the `ReturnDataTooLarge()` selector in `CallFailed.returnData` can also be returned by the target itself. Integrations should then rely on the bitmap rather than on error selectors. Alternatively, wrapping target failures in a dedicated error, such as `CallReverted(uint256 nonce, bytes returnData)` in strict mode, would make them distinguishable on-chain.

Remediation

This issue has been acknowledged by Paxos, and a fix was implemented in commit `0792f0ee` ↗.

4. Assessment Results

During our assessment on the scoped Bitmap Nonce Delegate contracts, we discovered three findings, all of which were informational in nature.

4.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.